Assignment_0_AralCimcim_k11720457

Machine Learning: Supervised Techniques WS24 Assignment 0

Within this introductory assignment we want to give you the chance to familiarize yourself with Jupyter Notebooks and this course's requirements for your submissions.

Copyrighting and Fare Use

This material, no matter whether in printed or electronic form, may be used for personal and non-commercial educational use only. Any reproduction of this material, no matter whether as a whole or in parts, no matter whether in printed or in electronic form, requires explicit prior acceptance of the authors.

Automatic Testing Guidelines

Your submissions are tested for correctness and plagiarism automatically using both visible and invisible assert statements. Automatic unittesting requires you, as a student, to submit a notebook which contains strictly defined objects.

Within the notebook we provide detailed instruction which you may want to follow, in order to maximise your final grade.

Please make sure to rerun your complete Notebook before submitting, this can reveal mistakes that could lead to point deductions!

LaTeX

During the course you will be often supposed to derive formulas and perform calculations.
The perfect tool to use in that case is LaTeX.
By developers definition: *"LaTeX is a high-quality typesetting system which includes features designed for the production of technical and scientific documentation."*

Tips how to use LaTeX:

- Enter LaTeX mode (inline) by typing iside of **two dollar signs.**
  Your first LaTeX statement $y = x^2$
- For centered, larger, stand-alone formulas use **double dollar signs** on each side: $$\sqrt{x^2} = y $$
- Use **curly brackets** to define arguments:
  No argument definition: $x^2 \cdot x^3 \neq x^2+3$
  Using brackets for argument definition: $x^2 \cdot x^3 = x^{2+3}$

- Go for fancy tags to make your styling readable.
  Blackboard Bold style can be used for sets of numbers. Use **\mathbb** tag.
  Examples: $\mathbb{R}, \mathbb{Z}, \mathbb{I}, \mathbb{Q}$
  Fractions can be written with **\frac** tag. Example: $\frac{x^2-1}{x-1}=\frac{(x-1)(x+1)}{x-1}=x+1$
- Adjust bracked hight with **\left \right** tags.
  Formula without brackets adjustments: $y = (\frac{\mathbb{E}(X)}{\mathbb{E}(Y)})^{x-y}$
  Formula with brackets adjustments: $y = \left(\frac{\mathbb{E}(X)}{\mathbb{E}(Y)}\right)^{x-y}$
- Full latex cheatsheet

Task 1: Write a LaTeX formula (7 points)

Show that for ax^2+bx+c roots are:$$x_{1,2}=\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

$-x^2+4x+5$ roots are:$$x_{1,2}=\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

$$x_{1,2}=\frac{-4\pm\sqrt{4^2-4 (-1 \cdot 5)}}{2 \cdot -1}$$

$$x_{1}={5}$$

$$x_{2}={-1}$$

Coding

In order to pass unittests please follow the instructions carefully.

Imports

Install and import packages you need.

In [31]:

```
import numpy as np
from matplotlib import pyplot as plt
```

Task 2: Write a function (5 points)

Write a function `bivar_normal()` which returns a numpy array of shape (1000,2)

filled with normally distributed random datapoints with `mean=0`, `std=0.01`.

In [32]:

```
# Do not change function names!
def bivar_normal():
    """Function bivar_normal takes no argument and returns a numpy array

    Returns
    -------
    np.ndarray
        numpy array of shape (1000,2) of np.float32 dtype with no rounding of values.
    """
    # YOUR CODE HERE
    array = np.random.normal(0, 0.01, (1000,2)).astype(np.float32)
    return array
```

In [10]:

```
assert bivar_normal().dtype == np.float32, "Wrong datatype!"
```

In [11]:

```
assert bivar_normal().shape == (1000, 2)
```

Task 3: Plot (5 points)

Define a variable `data` through calling the `bivar_normal()` function developed above.
Plot your `data` in a scatterplot by using the matplotlib library. Set axis labels and plot title properly.

In [23]:

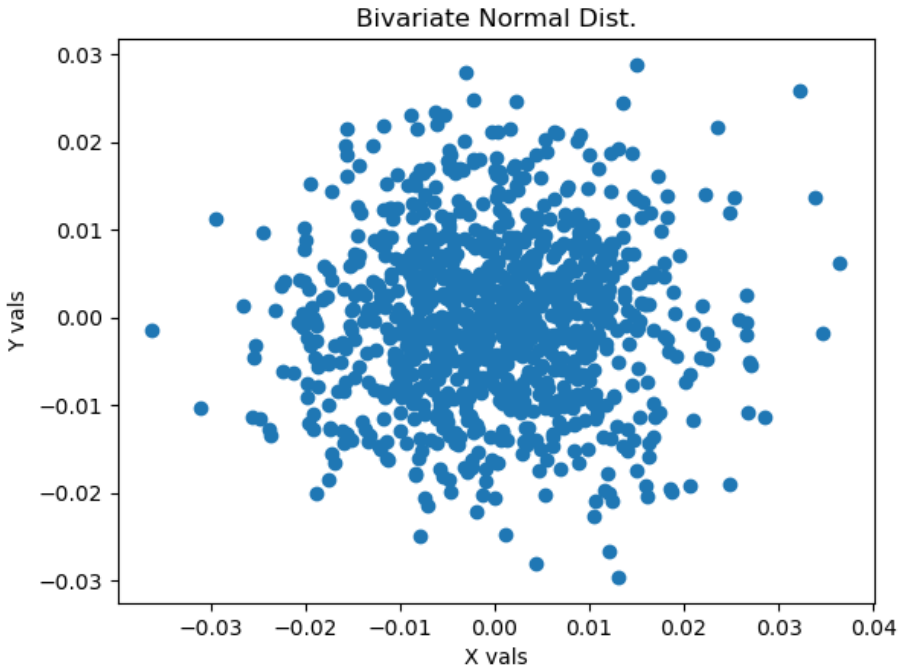
```
# Define `data` variable that stores the output of `bivar_normal()`
data = bivar_normal()
```

In [33]:

```
def gen_plot(data):
    """Write a function that plots the data."""
    # YOUR CODE HERE
    x = data[:,0]
    y = data[:,1]
    plt.scatter(x,y)
    plt.title("Bivariate Normal Dist.")
    plt.xlabel("X vals")
    plt.ylabel("Y vals")
    plt.show()
```

In [34]:

```
# Plot the data using your function
gen_plot(data)
```



Task 4: Test question (3 points)

What is true? Multiple answers might be correct.

Pegasus is:
a_) half horse, half human
b_) a horse with wings
c_) an elephant flying with its huge ears
d_) a mythical creature

To answer these questions, store the results in variables reflecting the actual question letters. For example, if you think that **a_)** is true, then create a variable called `a_` and assign the boolean value `True` to it: `a_ = True`. Do the same for the other options b_), c_) and d_).
Please **assign *boolean* values to all question variables**

In [26]:

```
# Examples for you (not the actual solution):
# a_ = True
```

```
# b_ = False
# c_ = True
# d_ = False

# YOUR CODE HERE
a_ = False
b_ = True
c_ = False
d_ = True
```

In [27]:

```
assert a_ is not None, "Store True/False!"
assert a_ in [True, False], "Invalid Answer!"
```

In [28]:

```
assert b_ is not None, "Store True/False!"
assert b_ in [True, False], "Invalid Answer!"
```

In [29]:

```
assert c_ is not None, "Store True/False!"
assert c_ in [True, False], "Invalid Answer!"
```

In [30]:

```
assert d_ is not None, "Store True/False!"
assert d_ in [True, False], "Invalid Answer!"
```